

Требования к проекту Cryptocurrency Investment Advisor Bot

Введение.....	3
Основные функции.....	3
Общие требования.....	4
Контроль версий и сотрудничество.....	4
Целевые артефакты.....	4
Функциональные требования.....	5
Аутентификация пользователей.....	5
Настройки взаимодействия.....	5
Взаимодействие с рыночными данными.....	6
Управление портфелем.....	6
История портфеля.....	6
Уведомления.....	7
Источник данных для курса криптовалют.....	7
Нефункциональные требования.....	8
Требования к технологическому стеку.....	10
Этапы выполнения проекта.....	11
Составление требований.....	11
Разработка архитектуры.....	11
Полный проект.....	11
Сценарий использования.....	12
UC-1. Аутентификация.....	12
UC-2. Настройка фиатной валюты.....	12
UC-3. Просмотр настроек фиатной валюты.....	13
UC-4. Добавление криптовалюты в отслеживаемые.....	13
UC-5. Удаление криптовалюты из отслеживаемых.....	14
UC-6. Просмотр отслеживаемых купюр.....	15
UC-7. Просмотр текущей цены криптовалюты.....	15
UC-8. Получение отчета об изменении цены отслеживаемых валют.....	16
UC-9. Сравнение цены двух криптовалют.....	17
UC-10. Добавление актива в портфель.....	17
UC-11. Удаление актива из портфеля.....	18
UC-12. Просмотр содержимого портфеля.....	18
UC-13. Просмотр стоимости портфеля.....	19
UC-14. Получение информации об изменении стоимости портфеля.....	19
UC-15. Получение информации об изменении стоимости каждой криптовалюты в портфеле.....	20
UC-16. Изменение пороговой цены для уведомления по криптовалюте.....	21
UC-17. Получение инвестиционного анализа по конкретной криптовалюте.....	21
UC-18. Получение обзора портфеля с рекомендациями LLM.....	23
UC-19. Запрос к LLM-консультанту в свободной форме.....	23
UC-20. Просмотр актуальных (сработавших) уведомлений.....	24

Глоссарий.....	25
Основные термины.....	25
Технические термины.....	26
Процесс разработки.....	27
Термины предметной области.....	28

Введение

Целью проекта является разработка Telegram-бота, который помогает пользователям следить за рынком криптовалют и принимать инвестиционные решения.

Основные функции

1. Настройка предпочтений: выбор криптовалюты по умолчанию.
2. Настройка предпочтений: выбор фиатной валюты по умолчанию.
3. Отображение текущей цены выбранной криптовалюты.
4. Отображение истории цены отслеживаемых криптовалют за указанный период.
5. Сравнение цен двух криптовалют.
6. Добавление актива в портфель (с указанием количества).
7. Удаление актива из портфеля (с указанием количества)
8. Вычисление стоимости портфеля в данный момент времени.
9. Отслеживание общей стоимости портфеля во времени.
10. Расчёт и отображение изменения стоимости по каждой криптовалюте в портфеле.
11. Получение от LLM инвестиционного анализа по конкретной криптовалюте.
12. Получение от LLM обзора портфеля с рекомендациями.
13. Запросы к LLM-советнику в свободной форме о крипторынке.
14. Установка ценовых алертов на определенные криптовалюты.
15. Автоматические уведомления при изменении цены более, чем на 5% для отслеживаемых криптовалют.
16. Автоматические уведомления при превышении ценой установленного значения для криптовалюты из отслеживаемых валют.

Общие требования

Контроль версий и сотрудничество

Весь код должен быть размещен в репозитории GitHub (<https://github.com/Java-Team-Spring-Framework/Cryptocurrency-Investment-Advisor-Bot>).

Репозиторий должен содержать README.md файл с подробным руководством по:

- Локальной настройке проекта.
- Сборке и запуску приложения.
- Развертыванию приложения с помощью Docker.
- Взаимодействию с ботом Telegram.

История изменений должна отражать значимый вклад всех членов команды.

Целевые артефакты

- Приложение должно создавать работающий файл fat JAR в качестве основного артефакта.
- Приложение должно быть доступно для развертывания в виде контейнера Docker.
- Для управления зависимостями, сборки и упаковки приложения используется Gradle.

Функциональные требования

Аутентификация пользователей

Идентификация происходит по Chat ID, дополнительная регистрация не требуется.

Настройки взаимодействия

- Выбор пользователем фиатной валюты для представления цены криптовалюты:
 - Если пользователь не указал фиатную валюту, добавлять фиатную валюту по умолчанию (доллары США).
 - Представление цены криптовалюты в виде фиатной валюты устанавливается единым образом для всех активов, используемых для отслеживания.
 - Выбор фиатной валюты предоставляется из заранее определенного списка, который включает евро, японскую иену, фунт стерлингов, турецкую лиру, российский рубль, китайский юань.
 - Если данная валюта уже выбрана, выдавать соответствующее сообщение и прекратить процесс выбора фиатной валюты.
- Просмотр текущей фиатной валюты для представления цены криптовалюты.
- Выбор пользователем криптовалют для отслеживания
 - Добавление криптовалюты в отслеживаемые
 - Если пользователь не добавил ни одну криптовалюту для отслеживания, то в отслеживаемых указывается криптовалюта по умолчанию (BTC).
 - Выбор криптовалют предоставляется из заранее определенного списка, который включает BTC, ETH, SOL, XRP, ADA, DOGE, AVAX, NEAR, LTC.
 - Если добавляемая валюта уже находится в отслеживании, выдавать соответствующее сообщение.
 - Удаление криптовалюты из отслеживаемых
 - Выбор криптовалюты для удаления предоставляется из списка отслеживаемых на данный момент криптовалют.

- Если после удаления в отслеживаемых не остается ни одной криптовалюты, то выдается соответствующее сообщение и в отслеживаемые добавляется криптовалюта по умолчанию (BTC).

- Просмотр текущего списка отслеживаемых криптовалют.
- Просмотр ответа на вопрос в свободной форме LLM-консультанту о криптовалютных рынках.
- Просмотр инвестиционного анализа на основе LLM для конкретной криптовалюты.

Взаимодействие с рыночными данными

- По запросу пользователя показывать текущую цену выбранной криптовалюты в ранее выбранной фиатной валюте.
- По запросу пользователя показывать отчет об изменении цены отслеживаемых криптовалют в фиатной валюте за указанный период :1 день, 7 дней, 30 дней.
- По запросу пользователя сравнивать цены на две выбранные криптовалюты.

Управление портфелем

- Добавление актива в портфель с указанием: тикера криптовалюты и количества монет
- Удаление актива из портфеля с указанием: тикера и количества монет. Выбор актива для удаления предоставляется из списка текущих активов в портфеле.
- Показывать содержимое портфеля с количеством монет и стоимости каждого актива.
- Вычислять общую стоимость портфеля на данный момент
- Отслеживать изменение стоимости портфеля за выбранный период: год, месяц, сутки
- Показывать изменение стоимости каждой криптовалюты в портфеле с момента первого добавления валюты в портфель.
- Получение обзора портфеля с рекомендациями LLM.

История портфеля

- Хранить следующую информацию о покупке актива:
- Дата в формате YYYY-MM-DD HH:MM:SS.
- Название криптовалюты.
- Цена в валюте по умолчанию.

Уведомления

Уведомления – сообщения, несущие информацию о изменениях в отслеживаемом активе, которые могут побудить пользователя к действиям на рынке (покупке или продаже).

- Посылать уведомления пользователю, когда:
 - Цена за единицу криптовалюты превышает установленное пользователем значение для данной криптовалюты.
 - Цена криптовалюты изменяется более, чем на 5% вверх или вниз за последние 24 часа. Уведомления начинают присылаться автоматически при добавлении криптовалюты в отслеживаемые.
- Формировать и выводить список актуальных уведомлений для пользователя на данный момент.
- Удалять уведомления из списка отправленных при потере актуальности уведомлений, то есть по истечении срока в 7 дней.

Источник данных для курса криптовалют

Сайт Binance (<https://www.binance.com/en>).

Нефункциональные требования

Производительность

- Запросы, которые зависят от внешних сервисов, должны выполняться неблокирующе: пользователь получает подтверждение приема запроса, а результат отправляется по мере готовности.
- Оптимизация базы данных: Структура базы данных PostgreSQL должна проектироваться с использованием необходимых индексов (например, по `chat_id`), чтобы обеспечивать быстрый поиск пользователей и их настроек без длительных блокировок.

Надежность

- Отказоустойчивость при работе с внешними API: При временной недоступности сторонних сервисов (API криптовалютной биржи Binance, LLM API, Telegram API) система должна использовать механизм повторных запросов (Retry) с возрастающей задержкой (например: 1с, 2с, 4с).
- Гарантия доставки: В случае сетевого сбоя при отправке уведомления пользователю, задача не должна теряться. Сообщение должно возвращаться в очередь брокера сообщений для повторной попытки отправки.

Безопасность

- Все конфиденциальные данные (токен Telegram-бота, ключи LLM API, учетные данные базы данных и брокера сообщений) должны передаваться исключительно через защищенные переменные окружения. Хранение ключей в исходном коде запрещено.
- Все данные, поступающие от пользователя (названия тикеров, числовые значения цен, объемы активов), должны проходить строгую валидацию по регулярным выражениям и диапазонам допустимых значений (например, отсутствие отрицательных сумм покупки).

Роли и доступы

В системе предусмотрены две разделенные роли:

1. Пользователь (User): Клиент системы, взаимодействующий с ней через интерфейс Telegram-бота. Имеет доступ только к своим рыночным предпочтениям, личному портфелю, истории запросов к LLM и своим уведомлениям. Идентификация происходит по `chat_id`.
2. Администратор (Admin): Пользователь с привилегированными правами, взаимодействующий с системой напрямую через вызовы HTTP API. Имеет доступ к системным метрикам и обезличенным данным всех пользователей.

Требования к авторизации

- Авторизация через Telegram (Роль: User):
 - Дополнительная регистрация с вводом логина/пароля не требуется.
 - При первой отправке команды `/start` система извлекает уникальный `chat_id` пользователя и создает новую запись в базе данных.
 - Все последующие команды бота проверяют наличие данного `chat_id` в системе. Если пользователя нет в БД, запрос отклоняется с предложением отправить `/start`.
- Авторизация через HTTP (Роль: Admin):
 - Осуществляется путем передачи токена доступа в HTTP-заголовке `Authorization` (формат: `Bearer <API_TOKEN>`).
 - Токен администратора должен быть надежно сгенерирован и задан на этапе развертывания через переменные окружения.

Конечные точки (HTTP Endpoints)

Сервер должен предоставлять следующие REST API эндпоинты для административного управления и мониторинга:

1. Проверка работоспособности системы

- Метод и путь: `GET /healthcheck`
- Авторизация: Не требуется (публичный доступ).
- Описание: Эндпоинт для мониторинга состояния приложения.
- Формат ответа: JSON-объект, содержащий текущий статус сервера (например, `{"status": "UP"}`) и массив с именами авторов-разработчиков проекта.

2. Получение списка пользователей

- Метод и путь: `GET /users`
- Авторизация: Строго требуется (только для роли Admin). Запросы без валидного заголовка `Authorization` должны получать статус `401 Unauthorized`.
- Описание: Возвращает информацию о зарегистрированных пользователях системы.
- Формат ответа: Массив JSON-объектов, содержащий базовые данные пользователей (внутренний ID, дата регистрации, установленные валюты по умолчанию и т.д.).

Требования к технологическому стеку

- Telegram API: Приложение должно быть интегрировано с Telegram API для обеспечения функциональности бота.
- LLM API: Приложение должно быть интегрировано с LLM API для обеспечения функциональности на базе искусственного интеллекта (OpenRoute).
- Язык программирования: Java 25
- Фреймворк: Spring 7 (ReactiveSpring (WebFlux), JOOQ, Spring Rest Docs)
- База данных: PostgreSQL 18.3
- Контейнеризации приложения: Docker, Docker Compose
- Логирование: SLF4J

Этапы выполнения проекта

Составление требований

Предоставить документ с требованиями, который включает:

- Цель и назначение бота.
- Основные функции.
- Нефункциональные требования.
- Ограничения.

Разработка архитектуры

Подготовить документ, описывающий архитектуру приложения:

- описание основных компонентов системы;
- диаграммы компонентов, блок-схемы;
- обоснование выбора технологического стека;
- описание процесса сборки, запуска и развертывания приложения;

Полный проект

Предоставить полный проект в виде:

- Ссылки на репозиторий GitHub с полной кодовой базой.
- Ссылки на Docker Hub, где опубликован Docker-образ.
- Ссылки или имя пользователя Telegram-бота для тестирования и взаимодействия.

Сценарий использования

UC-1. Аутентификация	
Триггер	Команда /start
Основной сценарий	1. Пользователь отправляет команду /start. 2. Бот устанавливает фиатную валюту по умолчанию, в отслеживаемые добавляется криптовалюта по умолчанию. 3. Выводится информация об установлении данных по умолчанию.

UC-2. Настройка фиатной валюты	
Триггер	Команда /set_fiat
Основной сценарий	1. Пользователь отправляет команду /set_fiat 2. Бот выводит сообщение о предложении выбора валюты. 3. Пользователю предлагаются для выбора кнопки с доступными фиатными валютами или кнопки назад. 4. Пользователь выбирает новую валюту из предложенных. 5. Бот меняет старую валюту на новую и выводит информацию о смене фиатной валюты.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирают валюту, которая уже установлена. 5. Бот выводит информацию о том, что данная валюта уже установлена.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс настройки фиатной валюты.

UC-3. Просмотр настроек фиатной валюты	
Триггер	Команда /current_fiat
Основной сценарий	1. Пользователь отправляет команду /current_fiat 2. Бот выводит информацию о текущей фиатной валюте.

UC-4. Добавление криптовалюты в отслеживаемые	
Триггер	Команда /add_tracked_crypto
Основной сценарий	1. Пользователь отправляет команду /add_tracked_crypto 2. Бот выводит сообщение о предложении выбора криптовалюты для добавления. 3. Пользователю предлагаются для выбора кнопки с доступными криптовалютами или кнопки “назад”. 4. Пользователь выбирает криптовалюту. 5. Бот подтверждает, что криптовалюта еще нет в отслеживаемых. 6. Бот отправляет сообщение с просьбой ввести цену в фиатной валюте, при превышении которой будет отправляться уведомление. 7. Пользователь вводит численное значение цены в фиатной валюте. 8. Бот проверяет корректность ввода и сохраняет данные отслеживаемой криптовалюты. 9. Выводится сообщение о добавлении криптовалюты и перечень криптовалют в отслеживаемых.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает криптовалюту, которая уже есть в отслеживаемых. 5. Бот выводит информацию о том, что данная валюта уже есть в отслеживаемых. Также выводится значение пороговой цены данной валюты.
Альтернативный сценарий	1-6 Основной сценарий 7. Пользователь вводит не численное значение. 8. Бот выводит информацию о том, что введена некорректная валюта, и не добавляет новую запись в

UC-4. Добавление криптовалюты в отслеживаемые	
	отслеживаемые. 9. Бот прекращает процесс добавления новой криптовалюты.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс добавления криптовалюты в отслеживаемые.

UC-5. Удаление криптовалюты из отслеживаемых	
Триггер	Команда /remove_tracked_crypto
Основной сценарий	1. Пользователь отправляет команду /remove_tracked_crypto 2. Бот выводит информационное сообщение о порядке удаления криптовалюты из отслеживаемых. 3. Пользователю предоставляются кнопки с отслеживаемыми криптовалютами или кнопка “назад”. 4. Пользователь выбирает валюту для удаления из отслеживаемых. 5. Бот удаляет выбранную валюту из списка отслеживаемых и проверяет, что в отслеживаемых еще остались криптовалюты. 6. Бот выводит обновленный список отслеживаемых криптовалют
Альтернативный сценарий	1-4 Основной сценарий 5. Бот после проверки определяет, что в отслеживаемых не осталось ни одной криптовалюты. 6. Бот добавляет в отслеживаемые криптовалюту по умолчанию. Бот выводит сообщение о том, что в списке не осталось криптовалют и была добавлена криптовалюта по умолчанию.

UC-5. Удаление криптовалюты из отслеживаемых	
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс удаления криптовалюты из отслеживаемых.

UC-6. Просмотр отслеживаемых купюр	
Триггер	Команда /tracked
Основной сценарий	1. Пользователь отправляет команду /current_tracked_crypto 2. Бот выводит текущий список отслеживаемых криптовалют и установленные для них пороговые цены.

UC-7. Просмотр текущей цены криптовалюты	
Триггер	Команда /price_crypto
Основной сценарий	1. Пользователь отправляет команду /price_crypto 2. Бот выводит информационное сообщение о выборе криптовалюты для просмотра цены. 3. Пользователю предлагаются кнопки с доступными криптовалютами или кнопка “назад”. 4. Пользователь выбирает валюту из списка. 5. Бот запрашивает актуальные данные на текущую цену выбранной криптовалюты в текущей фиатной валюте.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс просмотра текущей цены криптовалюты.

UC-8. Получение отчета об изменении цены отслеживаемых валют	
Триггер	Команда /price_history
Основной сценарий	1. Пользователь отправляет команду /price_history 2. Бот выводит информационное сообщение и предлагает кнопками выбор периода для отслеживаний : 1день, 7 дней, 30 дней или кнопка “назад”. 3. Пользователь выбирает период. 4. Бот формирует и выводит отчет об изменении цены в фиатной валюте за указанный период
Альтернативный сценарий	1-2 Основной сценарий 3. Пользователь выбирает кнопку “назад” 4. Бот прекращает процесс вывода отчета.

UC-9. Сравнение цены двух криптовалют	
Триггер	Команда /compare
Основной сценарий	1. Пользователь отправляет команду /compare 2. Бот отправляет информационное сообщение о выборе криптовалют для сравнения. 3. Пользователю предоставляется выбор кнопок с доступными криптовалютами для первой валюты или кнопка “назад”. 4. Пользователь выбирает первую криптовалюты. 5. Бот выводит информацию о необходимости выбора второй криптовалюты. 6. Пользователю предоставляется выбор кнопок с доступными криптовалютами для второй валюты или кнопка “назад”. 7. Пользователь выбирает вторую криптовалюту. 8. Бот запрашивает цены актуальные цены на данные криптовалюты в фиатной валюте. 9. Бот возвращает результат сравнения двух криптовалют.
Альтернативный	1-3 Основной сценарий

UC-9. Сравнение цены двух криптовалют	
сценарий	4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс сравнения криптовалют.
Альтернативный сценарий	1-6 Основной сценарий 7. Пользователь выбирает кнопку “назад” 8. Бот прекращает процесс сравнения криптовалют.

UC-10. Добавление актива в портфель	
Триггер	Команда /portfolio_add
Основной сценарий	1. Пользователь отправляет команду /portfolio_add 2. Бот выводит информационное сообщение о порядке добавления криптовалюты в портфель. 3. Пользователю предлагаются кнопки для выбора из доступных криптовалют или кнопка “назад”. 4. Пользователь выбирает криптовалюту для добавления. 5. Бот отправляет сообщение с просьбой ввести количество купленных монет. 6. Пользователь вводит численное значение. 7. Бот проверяет ввод количества монет. 8. Бот добавляет в портфель данные о криптовалюте, количестве монет и текущей цене криптовалюты при покупке. 9. Выводится сообщение об успешном добавлении и текущий состав портфеля.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс добавления актива в портфель.
Альтернативный сценарий	1-5 Основной сценарий 6. Пользователь вводит отрицательное или нечисловое значение. 7. Бот выдает сообщение о некорректном вводе и прекращает процесс добавления актива.

UC-11. Удаление актива из портфеля	
Триггер	Команда /portfolio_remove
Основной сценарий	1. Пользователь отправляет команду /portfolio_remove 2. Бот выводит информационное сообщение о порядке удаления криптовалюты из портфеля. 3. Пользователю предлагаются кнопки для выбора из доступных криптовалют в портфеле или кнопка “назад”. 4. Пользователь выбирает криптовалюту для удаления. 5. Бот отправляет сообщение с текущим количеством монет данной криптовалюты и просьбой ввести количество убираемых монет. 6. Пользователь вводит численное значение. 7. Бот проверяет ввод количества монет. 8. Бот убавляет количество монет данной криптовалюты на указанное значение. 9. Выводится сообщение об успешном удалении и текущий состав портфеля.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс удаления актива.
Альтернативный сценарий	1-5 6. Пользователь вводит отрицательное, нечисловое значение или значение меньше количества монет. 7. Бот выдает сообщение о некорректном вводе и прекращает процесс удаления актива.
Альтернативный сценарий	1-8 9. Количество монет данной криптовалюте стало равно нулю. 10. Из портфеля удаляется запись о данной криптовалюте.

UC-12. Просмотр содержимого портфеля	
Триггер	Команда /portfolio
Основной сценарий	1. Пользователь отправляет команду /portfolio

UC-12. Просмотр содержимого портфеля	
	- Бот проверяет, что портфель не пустой. - Бот выводит информацию о криптовалютах, их количестве и стоимости.
Альтернативный сценарий	1-2 - Портфель пустой. - Бот выводит информацию о том, что портфель пустой.

UC-13. Просмотр стоимости портфеля	
Триггер	Команда /portfolio_amount
Основной сценарий	- Пользователь отправляет команду /portfolio_amount. - Бот проверяет, что портфель не пустой - Бот вычисляет стоимость портфеля в текущий момент. - Вывод сообщения о стоимости портфеля в текущей фиатной валюте.
Альтернативный сценарий	1-2 - Портфель пустой. - Бот выводит информацию о том, что портфель пустой.

UC-14. Получение информации об изменении стоимости портфеля	
Триггер	Команда /portfolio_history
Основной сценарий	- Пользователь отправляет команду /portfolio_history. - Бот проверяет, что портфель не пустой. - Пользователю предлагается выбор кнопок с периодом отслеживаний - 1 день, 1 месяц, 1 год или кнопка “назад”. - Пользователь выбирает период. - Бот вычисляет разность стоимости портфеля за

UC-14. Получение информации об изменении стоимости портфеля	
	указанный период. 6. Вывод информации об изменении стоимости портфеля в указанной фиатной валюте.
Альтернативный сценарий	1-2 3. Портфель пустой. 4. Бот выводит информацию о том, что портфель пустой.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс вычисления изменения стоимости портфеля.

UC-15. Получение информации об изменении стоимости каждой криптовалюты в портфеле	
Триггер	Команда /portfolio_crypto_history
Основной сценарий	1. Пользователь отправляет команду /portfolio_crypto_history 2. Бот проверяет, что портфель не пустой. 3. Бот вычисляет изменение стоимости каждой криптовалюты с первого момента добавления валюты в портфель по текущий момент. 4. Выводится результат изменения стоимости криптовалют в фиатной валюте.
Альтернативный сценарий	1-2 3. Портфель пустой. 4. Бот выводит информацию о том, что портфель пустой.

UC-16. Изменение пороговой цены для уведомления по криптовалюте	
Триггер	Команда /set_alert
Основной сценарий	1. Пользователь отправил команду /set_alert. 2. Бот выводит перечень отслеживаемых криптовалют и пороговые цены для каждой криптовалюты. 3. Пользователю предоставляются кнопки для выбора криптовалюты для изменения пороговой цены или кнопка “назад”. 4. Пользователь выбирает криптовалюту. 5. Бот выводит сообщение с просьбой ввести новую пороговую цену для криптовалюты. 6. Пользователь вводит новое значение. 7. Бот проверяет введенное значение. 8. Бот изменяет значение пороговой цены для уведомлений по данной криптовалюте и выводит соответствующее сообщение.
Альтернативный сценарий	1-3 Основной сценарий 4. Пользователь выбирает кнопку “назад” 5. Бот прекращает процесс вычисления изменения стоимости портфеля.
Альтернативный сценарий	1-5 6. Пользователь вводит отрицательное, нечисловое значение. 7. Бот выдает сообщение о некорректном вводе и прекращает процесс удаления актива.

UC-17. Получение инвестиционного анализа по конкретной криптовалюте	
Триггер	Команда /llm_analyze
Основной сценарий	1. Пользователь отправляет команду /llm_analyze.

UC-17. Получение инвестиционного анализа по конкретной криптовалюте

	2. Бот выводит информационное сообщение с предложением выбрать криптовалюту для анализа. 3. Пользователю предлагаются кнопки с доступными криптовалютами или кнопка "назад". 4. Пользователь выбирает криптовалюту. 5. Бот отправляет сообщение о том, что запрос обрабатывается (ожидание ответа от нейросети). 6. Бот формирует промпт и отправляет запрос к LLM API. 7. Бот получает ответ и выводит пользователю готовый инвестиционный анализ по выбранной монете.
Альтернативный сценарий	1-3. Основной сценарий. 4. Пользователь выбирает кнопку "назад". 5. Бот прекращает процесс получения анализа.
Альтернативный сценарий	1-5. Основной сценарий. 6. Внешний сервис LLM API недоступен или возвращает ошибку. 7. Бот использует механизм повторных запросов (Retry). Если попытки исчерпаны, выводит сообщение о временной недоступности сервиса ИИ и предлагает повторить позже.

UC-18. Получение обзора портфеля с рекомендациями LLM	
Триггер	Команда <code>/llm_portfolio</code>
Основной сценарий	1. Пользователь отправляет команду <code>/llm_portfolio</code>. 2. Бот проверяет, что портфель не пустой. 3. Бот собирает текущие данные портфеля (состав, количество, текущая стоимость, изменение цены). 4. Бот отправляет сообщение о том, что анализ портфеля генерируется. 5. Бот передает агрегированные обезличенные данные портфеля в LLM API с запросом на ревью и рекомендации. 6. Бот выводит пользователю ответ от LLM-консультанта.
Альтернативный сценарий	1-2. Основной сценарий. 3. Портфель пустой. 4. Бот выводит информацию о том, что для получения рекомендаций необходимо сначала добавить активы в портфель.

UC-19. Запрос к LLM-консультанту в свободной форме	
Триггер	Команда <code>/llm_ask</code>
Основной сценарий	1. Пользователь отправляет команду <code>/llm_ask</code>

UC-19. Запрос к LLM-консультанту в свободной форме	
	2. Бот выводит сообщение с просьбой написать свой вопрос о криптовалютном рынке. 3. Пользователь отправляет текстовое сообщение с вопросом. 4. Бот отправляет сообщение о том, что запрос обрабатывается. 5. Бот перенаправляет текст пользователя в LLM API. 6. Бот выводит пользователю полученный ответ от ИИ.
Альтернативный сценарий	1-2. Основной сценарий. 3. Пользователь передумал и отправляет команду /cancel (или нажимает "назад", если предусмотрена кнопка). 4. Бот прекращает режим ожидания вопроса.

UC-20. Просмотр актуальных (сработавших) уведомлений	
Триггер	Команда /alerts_history (или /alerts)
Основной сценарий	1. Пользователь отправляет команду /alerts. 2. Бот обращается к базе данных и запрашивает историю сработавших уведомлений для данного пользователя за последние 7 дней (согласно требованиям). 3. Бот проверяет, есть ли в списке записи. 4. Бот формирует и выводит пользователю список сработавших алертов с указанием даты, времени, криптовалюты и причины (например: "Превышен порог цены" или "Изменение цены > 5%").

UC-20. Просмотр актуальных (сработавших) уведомлений

Альтернативный сценарий	1-3. Основной сценарий. 4. В базе данных нет сработавших уведомлений за последние 7 дней. 5. Бот выводит сообщение о том, что актуальных уведомлений за последнее время не поступало.
-------------------------	--

Глоссарий

Основные термины

Telegram	кроссплатформенный мессенджер, в котором пользователи обмениваются сообщениями и взаимодействуют с ботами.
Telegram-бот	программное приложение, работающее внутри Telegram и автоматически обрабатывающее команды и сообщения пользователей.
Chat ID	уникальный числовой идентификатор пользователя или чата в Telegram, используемый системой для идентификации и аутентификации пользователя.
Пользователь	лицо, взаимодействующее с ботом через Telegram и использующее его функции для получения информации о криптовалютах, управления портфелем и настройки уведомлений.
Администратор	пользователь с расширенными правами доступа, который

	взаимодействует с системой через HTTP API и имеет доступ к административным функциям.
Криптовалюта	цифровой актив, использующий криптографические методы для обеспечения безопасности транзакций и учета владения.
Фиатная валюта	государственная валюта, используемая для отображения стоимости криптовалют и результатов расчетов в приложении.
Курс криптовалюты	текущая стоимость криптовалюты, выраженная в выбранной фиатной валюте.
Портфель	совокупность криптовалютных активов пользователя с указанием количества и цены покупки, используемая для расчета текущей стоимости, прибыли и убытка.
Алерт	пользовательское уведомление о наступлении определенного рыночного события, например достижения заданной цены или сильного изменения курса криптовалюты.
LLM-консультант	модуль системы, использующий LLM API для генерации инвестиционного анализа.

Технические термины

Git	распределенная система контроля версий, применяемая для отслеживания изменений в коде и организации совместной разработки
GitHub	интернет-платформа для хранения репозитория и совместной разработки проекта.
Репозиторий GitHub	хранилище исходного кода проекта, истории изменений и сопроводительной документации.
README.md	файл с инструкциями по локальной настройке проекта, сборке, запуску, развертыванию через Docker и взаимодействию с Telegram-ботом.
Fat JAR	исполняемый JAR-файл, содержащий приложение и все его зависимости, необходимый для запуска системы как основного артефакта проекта.

API	набор правил и методов, по которым различные программные компоненты взаимодействуют друг с другом и обмениваются данными.
Telegram API	интерфейс, позволяющий приложению взаимодействовать с платформой Telegram.
LLM API	интерфейс для взаимодействия с большой языковой моделью.
Spring Framework	фреймворк для разработки Java-приложений, используемый для организации структуры приложения, внедрения зависимостей и реализации HTTP API
Reactive Spring (WebFlux)	реактивный модуль Spring для построения неблокирующих веб-приложений и API.
Spring Rest Docs	инструмент документирования HTTP API на основе тестов и фактических ответов приложения.
PostgreSQL	реляционная база данных, используемая для хранения данных.
RabbitMQ	брокер сообщений, обеспечивающий асинхронное взаимодействие компонентов системы, например при обработке уведомлений и фоновых задач.
Аутентификация	процесс установления личности пользователя.
Авторизация	процесс проверки прав доступа пользователя к определенным функциям системы после успешной аутентификации

Процесс разработки

Логирование	процесс записи событий и действий приложения для мониторинга, диагностики ошибок и анализа работы системы.
Валидация	проверка входных данных на корректность и соответствие заданным ограничениям.
Развертывание	процесс подготовки и запуска приложения в целевом окружении.

Gradle	система сборки проекта
Docker	платформа для контейнеризации приложения и его зависимостей, обеспечивающая переносимость и единообразие окружения.
Docker Compose	инструмент для запуска многоконтейнерного приложения, позволяющий поднимать связанные сервисы, например приложение, базу данных и брокер сообщений, в локальной среде.
Docker-образ	шаблон приложения, на основе которого создается Docker-контейнер для развертывания системы.
JUnit	библиотека для модульного тестирования Java-приложений, предусмотренная в требованиях к проекту

Термины предметной области

Криптовалюты:

BTC	Bitcoin
ETH	Ethereum
SOL	Solana
XRP	Ripple
ADA	Cardano
DOGE	Dogecoin
AVAX	Avalanche
NEAR	NEAR Protocol
LTC	Litecoin

Фиатные валюты:

USD	Доллар США
EUR	Евро
JPY	Японская иена

GBP	Фунт стерлингов
RUB	Российский рубль
CNY	Китайский юань